

Release plan: Claude Bootstrap Hardening

Общая цель релиза

Добавить в claude-bootstrap несколько уровней защиты от опасных операций:

Rules

↓

Native Claude Code permissions

↓

Context-aware PreToolUse/PostToolUse hooks

↓

Optional Claude Code Bash sandbox configuration

↓

Optional external guard follow-up

Релиз не должен превращать claude-bootstrap в полноценный shell security engine.

Основной

scope – небольшой набор надёжных правил, понятные подтверждения и безопасное управление project settings.

Architecture alignment note

Tasks 1-17 are implemented in code and must stay aligned with this responsibility split:

Rules guide behavior, permissions are native static ask/deny controls, hooks provide project-specific contextual checks, Claude Code owns command matching and sandbox runtime, and external guard integration is outside the main hardening release. Any earlier mention of external guard support in Tasks 1-17 is superseded by Task 22 as an optional follow-up.

Task 1. Добавить rule для destructive operations

Цель

Добавить объясняющий слой безопасности в .claude/rules/ .

Это не техническая блокировка, а инструкция для Claude о том, как обрабатывать потенциально опасные операции.

Изменения

Создать:

plugin/rules/common/destructive-operations.md

Добавить правила:

Destructive Operations

- Never discard uncommitted changes without explicit user approval.
- Prefer `git status`, `git diff`, dry-run and backup commands first.

- Never deploy, publish, force-push or modify production resources implicitly.

- When a destructive action is necessary, explain its impact and request approval.

- Never bypass repository hooks or CI checks.

- Treat production, credential and infrastructure operations as high risk.

Дополнить документ разделами:

- Git operations;
- filesystem operations;
- infrastructure operations;
- database operations;
- package publication;
- secrets;
- preferred safe alternatives.

Примеры альтернатив:

git push --force

→ git push --force-with-lease

terraform apply

→ terraform plan

kubectl delete

→ kubectl get/describe

curl URL | sh

→ download, inspect, execute

Дополнительные изменения

Обновить:

- /bootstrap ;
- /doctor ;
- README;
- ожидаемое количество common rules в тестах и документации.

Не входит в задачу

- permissions;
- hooks;
- command parsing;
- sandbox.

Критерии приёмки

- Новый rule копируется при /bootstrap .
- /bootstrap --update обновляет его.
- /doctor видит новый rule.
- README содержит описание.
- Повторный bootstrap не создаёт дубликаты.

Оценка

2–3 story points

Task 2. Добавить baseline permissions profile

Цель

Подготовить статический project-level preset с безопасными permissions.ask и permissions.deny .

На данном этапе profile ещё не устанавливается через /harden . Он существует как валидируемый шаблон.

Изменения

Создать:

plugin/hardening/profiles/baseline.settings.json

Базовый состав:

```
{
  "permissions": {
    "disableBypassPermissionsMode": "disable",
    "deny": [
      "Read(../.env)",
      "Read(../.env.local)",
      "Read(../.env.production)",
    ]
  }
}
```

```

"Read(~/.ssh/**)",
"Read(~/.aws/credentials)",
"Read(~/.kube/config)",
"Bash(gh repo delete *)",
"Bash(git stash clear *)",
"Bash(npm unpublish *)"
],
"ask": [
"Bash(npm publish *)",
"Bash(pnpm publish *)",
"Bash(yarn npm publish *)",
"Bash(cargo publish *)",
"Bash(twine upload *)",
"Bash(gh release create *)",
"Bash(gh release delete *)",
"Bash(terraform apply *)",
"Bash(terraform destroy *)",

"Bash(kubectl delete *)",
"Bash(helm uninstall *)"
]
}
}

```

Ограничения

Не добавлять широкие правила:

```

Bash(rm *)
Bash(git reset *)
Bash(git clean *)
Bash(sudo *)

```

Они требуют анализа контекста и должны обрабатываться hook-guard.

Тесты

Добавить проверку:

- JSON синтаксически корректен;
- deny и ask не содержат дубликатов;
- отсутствуют запрещённые широкие patterns;
- каждое правило имеет допустимый формат.

Не входит в задачу

- автоматическое применение profile;
- merge с существующим settings;
- strict profile;
- sandbox.

Критерии приёмы

- Profile валиден.
- CI проверяет его.
- README объясняет назначение baseline permissions.
- Profile не содержит широких deny для контекстных команд.

Оценка

2–3 story points

Task 3. Реализовать безопасный merge

.claude/settings.json

Цель

Создать отдельный инструмент, который безопасно объединяет hardening profile с существующим project settings.

Изменения

Создать:

plugin/hardening/apply_profile.py
tests/test_apply_profile.py

Поддерживаемые команды

```
python apply_profile.py --profile baseline
python apply_profile.py --profile baseline --dry-run
python apply_profile.py --profile baseline --check
python apply_profile.py --profile baseline --force
```

Требования к merge

Инструмент должен:

- сохранять неизвестные ключи;
- сохранять пользовательские hooks;
- объединять allow, ask и deny ;
- удалять дубликаты;
- сохранять порядок существующих entries;
- использовать atomic write;
- создавать backup только при реальном изменении;
- не менять formatting при повторном запуске;
- быть идемпотентным.

Scalar conflicts

Для полей вроде:

```
permissions.disableBypassPermissionsMode
sandbox.enabled
```

использовать правила:

значение отсутствует

→ добавить

значение совпадает

→ оставить

значение отличается

→ сообщить conflict

--force

→ разрешить замену

Сценарии тестирования

- settings отсутствует;
- settings существует;
- существующие permissions;
- пользовательские hooks;
- дублирующиеся entries;
- conflicting scalar;
- invalid JSON;
- dry-run;
- check;

- повторное применение;
- ошибка записи;
- backup;
- atomic rollback.

Не входит в задачу

- /harden ;
- command guard;
- sandbox;
- удаление ранее применённого profile.

Критерии приёмки

- Пользовательские settings не теряются.
- Повторный запуск не изменяет файл.
- Dry-run показывает diff без записи.
- Conflict не перезаписывается без --force .
- Все merge-сценарии покрыты unit tests.

Оценка

5 story points

Task 4. Добавить project security policy и JSON

schema

Цель

Вынести контекстные настройки guard из Python-кода в project-level policy.

Изменения

Создать:

plugin/hardening/security-policy.schema.json

plugin/hardening/defaults/baseline-policy.json

Пример policy:

```
{
  "version": 1,
  "managedBy": "claude-bootstrap",
  "profile": "baseline",
  "protectedBranches": [
    "main",
    "master"
  ],
  "production": {
    "markers": [
      "prod",
      "production"
    ],
    "kubeContexts": [],
    "terraformWorkspaces": [],
    "awsAccountIds": []
  },
  "paths": {
    "safeRecursiveDelete": [
      ".cache",
      "dist",
      "build",
      "node_modules",
      "tmp"
    ],
    "generated": [
      "**/generated/**",

```

```
"/**/*.generated.*",
"/**/vendor/**"
]
```

```
}
"largeFiles": {
  "warningLines": 800,
  "askOnCreateLines": 1200,
  "askOnGrowthLines": 100
}
```

Требования к schema

Schema должна:

- требовать поле version ;
- запрещать неизвестные top-level поля;
- валидировать типы;
- запрещать отрицательные thresholds;
- валидировать решения;
- содержать descriptions;
- поддерживать versioned format.

Интеграция

apply_profile.py должен уметь:

- создавать .claude/security-policy.json ;
- не перезаписывать существующий project policy;
- проверять policy через schema;
- показывать ошибку до изменения settings.

Не входит в задачу

- чтение policy command guard;
- production detection;
- strict policy.

Критерии приёмы

- Baseline policy создаётся в проекте.
- Existing policy сохраняется.
- Некорректный policy отклоняется.
- CI валидирует default policy.
- Schema покрыта позитивными и негативными тестами.

Оценка

3–5 story points

Task 5. Исправить block-no-verify без внедрения общего guard

Цель

Устранить существующие false positives как самостоятельное исправление.

Проблема

Текущий hook ищет --no-verify во всей Bash-строке и может заблокировать:

```
echo "Do not use --no-verify"
grep -- "--no-verify" README.md
some-unrelated-tool --no-verify
```

Изменения

Исправить block-no-verify.sh , чтобы он блокировал только:

```
git commit ... --no-verify
git push ... --no-verify
Также блокировать:
```

```
HUSKY=0 git commit
HUSKY=0 git push
SKIP=... git commit
SKIP=... git push
```

Поддержать:

- флаги до и после аргументов;
- leading environment assignments;
- обычные compound commands;
- quoted commit messages.

Тесты

Добавить fixtures:

```
git commit --no-verify
git --no-pager commit --no-verify
git push origin main --no-verify
```

```
echo "--no-verify"
grep -- "--no-verify"
git log --grep="--no-verify"
some-tool --no-verify
```

Не входит в задачу

- Python guard;
- ask ;
- force-push policy;
- branch detection.

Критерии приёмки

- Git commit/push bypass блокируется.
- Нерелевантные команды не блокируются.
- Существующий hook contract сохраняется.
- Tests проходят на Linux и macOS.

Оценка

2-3 story points

Task 6. Создать Python command guard foundation

Цель

Добавить общий Python entrypoint и внутреннюю модель решений, но пока без большого набора security rules.

Изменения

Создать:

```
plugin/hooks/scripts/command_guard.py
plugin/hooks/guard/__init__.py
plugin/hooks/guard/decisions.py
plugin/hooks/guard/context.py
plugin/hooks/guard/shell.py
plugin/hooks/guard/rules.py
tests/test_command_guard.py
tests/fixtures/bash-commands.json
```

Поддерживаемые решения

deny

ask

warning/additionalContext

no decision

Приоритет:

deny > ask > warning > no decision

Input

Guard должен читать hook JSON из stdin.

Минимально использовать:

```
{
  "hook_event_name": "PreToolUse",
  "tool_name": "Bash",
  "tool_input": {
    "command": "git reset --hard"
  },
  "cwd": "/project"
}
```

Output

Deny:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "[RULE-ID] Explanation."
  }
}
```

Ask:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "ask",

    "permissionDecisionReason": "[RULE-ID] Confirmation is required."
  }
}
```

Shell parsing scope

Поддержать:

- && ;
- || ;
- ; ;
- pipe;
- newline;
- single quotes;
- double quotes;
- backslash escaping;
- leading environment assignments;
- wrappers:
- sudo ;
- env ;

- `command` ;
 - `nohup` .
- Не реализовывать глубокий анализ:
- `$()` ;
 - `backticks`;
 - `heredoc`;
 - `bash -c` ;
 - `sh -c` ;
 - Python/Node inline code;
 - PowerShell AST.

Error handling

Baseline behavior при internal error:

- не возвращать `allow` ;
- вывести краткий `warning`;
- завершиться с кодом `0` ;
- оставить стандартный Claude permission flow.

Критерии приёмки

- Guard корректно читает и возвращает JSON.
- Decision model покрыта тестами.
- Parser не падает на malformed input.
- Unsupported constructs определяются отдельно.
- Entry point не имеет внешних Python dependencies.

Оценка

5 story points

Task 7. Добавить Git rules в command guard

Цель

Перенести Git security policy в контекстный Python guard.

Hard deny

Реализовать:

```
git push --force в protected branch
git push --force-with-lease в protected branch
git branch -D protected branch
git stash clear
git commit --no-verify
git push --no-verify
HUSKY=0 git commit/push
SKIP=... git commit/push
Protected branches читать из:
```

`.claude/security-policy.json`

Fallback:

```
main
master
```

Ask

Реализовать:

```
git reset --hard
git clean -f/-fd/-fdx
git restore с возможной потерей изменений
git checkout -- <path>
git stash drop
```

```
git push --force feature branch
git push --force-with-lease feature branch
```

Контекст

Guard должен определять:

- текущую branch;
- dirty working tree;
- project root.

Правила:

```
git reset --hard + dirty tree → deny
git reset --hard + clean tree → ask
protected branch + force push → deny
feature branch + --force
```

→ ask

feature branch + force lease

→ ask

unknown branch

→ ask

Subprocess restrictions

Разрешены только read-only команды:

```
git rev-parse --show-toplevel
git branch --show-current
git status --porcelain
```

Для каждого subprocess установить timeout.

Migration

После внедрения Git rules:

- удалить active registration block-no-verify.sh ;
- временно оставить script в репозитории как compatibility wrapper;
- обновить hooks tests.

Критерии приёмки

- Git decision matrix покрыта fixtures.
- Dirty working tree влияет на git reset --hard .
- Force push в protected branch блокируется.
- Feature branch force push требует подтверждения.
- Старые false positives не возвращаются.

Оценка

5 story points

Task 8. Добавить filesystem rules

Цель
Добавить безопасную обработку разрушительных filesystem-команд.

Hard deny

```
rm -rf /
rm -rf ~
rm -rf $HOME
rm -rf project root
```

```
rm -rf parent of project root
rm -rf .git
mkfs
wipefs
fdisk
parted
dd ... of=/dev/...
```

Ask

```
rm -rf обычной директории
recursive chmod
recursive chown
sudo
```

Path normalization

Перед решением:

- раскрывать ~ ;
- заменять \$HOME и \${HOME} ;
- нормализовать .. ;
- разрешать относительные пути от cwd;
- сравнивать resolved path с home и project root.

Не выполнять:

- command substitution;
- arbitrary environment expansion;

- glob expansion.

При unresolved variable, glob или \$() возвращать ask , а не deny .

Safe path policy

Пути из:

```
.cache
dist
build
node_modules
tmp
```

по умолчанию всё равно требуют ask .

Project policy в будущем сможет изменить это поведение.

Тесты

Покрыть:

- absolute paths;
- relative paths;
- paths with spaces;
- quoted paths;
- ../ ;
- unresolved variables;
- globs;
- symlink-like paths;
- project root;
- parent directory;
- home;
- /tmp ;
- build directories.

Критерии приёмки

- Критические filesystem operations блокируются.
- Обычное recursive deletion требует подтверждения.
- Нормализация путей не выполняет shell code.
- Parser failure не приводит к silent allow.

Оценка
5 story points

Task 9. Добавить infrastructure и publication rules

Цель

Добавить контекстные подтверждения для infrastructure, release и package operations.

Ask

```
terraform apply
terraform destroy
kubectl delete
helm uninstall
docker system prune
npm publish
pnpm publish
yarn npm publish
cargo publish
twine upload
gh release create
gh release delete
curl ... | sh
wget ... | sh
```

Hard deny

```
gh repo delete
npm unpublish
kubectl delete namespace kube-system
kubectl delete namespace default
destructive operation в подтверждённом production
```

Production detection

Использовать:

- command flags;
- ENV ;
- ENVIRONMENT ;
- STAGE ;
- NODE_ENV ;
- Kubernetes context;
- Terraform workspace;
- project policy markers.

Не выполнять сетевые команды для environment detection.

Разрешённые context commands:

```
kubectl config current-context
terraform workspace show
```

Оба – с timeout и обработкой отсутствующего CLI.

Правила

```
production confirmed → deny
dev/local
```

→ ask

unknown environment

→ ask

Remote scripts

Для:

curl URL | sh

wget URL | bash

возвращать ask с рекомендацией:

1. скачать;
2. проверить checksum;
3. просмотреть содержимое;
4. выполнить отдельно.

Критерии приёмы

- Publication требует подтверждения.
- Удаление repository блокируется.
- Production destructive operations блокируются.
- Unknown environment не считается безопасным.
- Отсутствие kubectl/terraform не ломает guard.

Оценка

5 story points

Task 10. Переработать large file policy

Цель

Заменить жёсткое ограничение 800 строк на градуированную policy.

Новое поведение

< 800 строк

→ no decision

800–1200 строк

→ warning

новый файл > 1200 строк

→ ask

существующий файл уменьшается

→ no block

существующий файл растёт <100 строк → warning

существующий файл растёт ≥100 строк → ask

excluded file

→ no decision

Thresholds брать из:

.claude/security-policy.json

Write

Для Write считать строки из:

tool_input.content

Edit

Для Edit учитывать:

- текущий файл;
- old_string ;
- new_string ;
- replace_all ;
- число совпадений.

Если точный итог вычислить невозможно:

warning

Не возвращать deny .

Исключения

generated

vendor

schemas

migrations

snapshots

lockfiles

minified files

Migration

После внедрения:

- удалить active registration block-large-files.sh ;
- обновить README;
- обновить tests;
- оставить старый script только как временный compatibility wrapper.

Критерии приёмки

- Большой существующий файл можно постепенно уменьшать.
- Новый огромный файл требует подтверждения.
- Generated files не создают шум.
- replace_all корректно учитывается.
- Ошибка вычисления не блокирует изменение.

Оценка

3–5 story points

Task 11. Добавить PreToolUse secret detection

Цель

Обнаруживать high-confidence secrets до записи файла.

Изменения

Создать:

plugin/hooks/guard/secrets.py

tests/fixtures/secret-patterns.json

Guard должен обрабатывать:

Write

Edit

High-confidence patterns

- private key block;
- AWS access key;
- GitHub PAT;
- GitLab PAT;
- Slack token;

- credential URL;
- длинный literal secret в key-like variable.

Решения

private key в source file

→ deny

PAT/AWS key в tracked source

→ deny

secret в ignored .env

→ ask

generic password/token assignment

→ ask

JWT или fixture-like token

→ warning или ask

placeholder/example value

→ no decision

Git context

Использовать read-only проверки:

git check-ignore

git ls-files

Нужно различать:

- tracked file;
- ignored file;
- untracked committable file.

Security requirements

Никогда не выводить:

- найденный secret;
- полный matched value;
- полный content.

Reason должен содержать:

- rule ID;

- тип secret;
- путь;
- remediation.

PostToolUse

На этом этапе допускается оставить существующий warn-secrets.sh как дополнительную

проверку после записи.

Его удаление и унификацию вынести в следующую задачу.

Критерии приёмки

- Private keys и PAT блокируются до записи.
- .env больше не пропускается безусловно.
- Test placeholders не создают постоянные false positives.

- Reason не содержит секрет.
- Tracked и ignored files обрабатываются по-разному.

Оценка
5 story points

Task 12. Унифицировать PostToolUse warnings

Цель

Убрать дублирование security regex и привести warning hooks к общей Python-инфраструктуре.

Изменения

Перенести в Python:

- post-write secret warning;
- debug code warning;
- generated-file warning;
- CI workflow change warning;
- migration change warning;
- TODO/FIXME warning.

Создать общий entrypoint либо расширить:

command_guard.py
поддержкой PostToolUse .

Важное ограничение

PostToolUse не должен пытаться блокировать уже выполненное изменение.

Разрешённые результаты:

- warning;
- additional context;
- no decision.

Общая база patterns

PreToolUse и PostToolUse secret detection должны использовать один модуль:

plugin/hooks/guard/secrets.py
Не поддерживать два независимых набора regex.

Не входит в задачу

- coverage regression;
 - lockfile/manifest semantic comparison;
 - dependency approval;
 - linter suppression analysis.
- Эти проверки требуют анализа repository diff или CI.

Migration

После внедрения удалить active registration:

warn-secrets.sh
warn-debug-code.sh
Compatibility wrappers можно оставить на один minor release.

Критерии приёмки

- Secret patterns не дублируются.
- PostToolUse никогда не возвращает deny.
- Debug warnings сохраняют прежнее поведение.
- CI workflow и migration changes создают warning.
- Чистые изменения не создают output.

Оценка
3–5 story points

Task 13. Перевести hook configuration на единый guard

Цель

Обновить plugin и manual hook configurations после готовности Python guard.

Изменения

Обновить:

plugin/hooks/hooks.json
plugin/settings-hooks.json

Plugin path

python3 "\$CLAUDE_PLUGIN_DIR/hooks/scripts/command_guard.py"

Manual installation path

python3 ~/.claude/hooks/scripts/command_guard.py

Matchers

Настроить:

PreToolUse:

Bash

Write|Edit

PostToolUse:

Write|Edit

Поле if использовать как предварительную оптимизацию для Bash-команд:

git

rm

sudo

curl

wget

terraform

kubect1

helm

docker

npm

pnpm

yarn

cargo

twine

gh

psql

mysql

sqlite3

prisma

alembic

mkfs

wipefs

fdisk

parted

dd

chmod

chown

Требования

- Один tool call не должен запускать один и тот же guard несколько раз.
- Не должно остаться active references на legacy security scripts.
- remind-compact.sh можно оставить отдельным hook.
- Пользовательские hooks не изменяются.

Тесты

- JSON validation;
- command path для plugin;
- command path для manual install;
- отсутствие legacy active references;
- отсутствие duplicate hook commands;
- PreToolUse и PostToolUse integration.

Критерии приёмки

- Plugin использует единый guard.
- Manual settings используют единый guard.
- Старые security hooks не запускаются.
- Новый guard получает Bash, Write и Edit payloads.
- Hook timeout задан явно.

Оценка

3 story points

Task 14. Добавить migration в installer и uninstaller

Цель

Обеспечить безопасное обновление существующих manual installations.

install.sh

Добавить:

- копирование Python guard;
- копирование hooks/guard ;
- копирование hardening assets;
- обнаружение legacy hook commands;
- удаление только legacy commands;
- добавление нового command guard;
- сохранение пользовательских hooks;
- backup;
- diff preview;
- проверку Python;
- warning при отсутствии Python.

Legacy commands:

block-no-verify.sh

block-large-files.sh

warn-secrets.sh

warn-debug-code.sh

uninstall.sh

Удалять:

- Python guard;
- hooks/guard ;
- hardening assets;
- /harden ;
- новые hook entries;
- legacy hook entries.

Не удалять project files:

```
project/.claude/settings.json
project/.claude/security-policy.json
project/.claude/rules/
```

Идемпотентность

Повторный `install.sh` не должен:

- дублировать hooks;
- повторно мигрировать отсутствующие entries;
- менять settings без необходимости.

Критерии приёмки

- Existing manual installation обновляется без потери custom hooks.
- Создаётся backup.
- Legacy entries удаляются.
- Новый hook добавляется один раз.
- Uninstall удаляет только глобальные installed assets.
- Project configuration остаётся нетронутой.

Оценка

5 story points

Task 15. Добавить `/harden --baseline`

Цель

Предоставить пользователю отдельную команду для применения baseline hardening к конкретному проекту.

Изменения

Создать:

`plugin/skills/harden/SKILL.md`

Поддерживаемые команды

`/harden`

`/harden --baseline`

`/harden --dry-run`

`/harden --check`

Default:

`/harden == /harden --baseline`

Алгоритм

1. Определить project root.
2. Найти hardening assets.
3. Проверить Python.
4. Прочитать существующие project settings.
5. Показать:
6. создаваемые файлы;
7. добавляемые permissions;
8. conflicts;
9. diff.
10. Запросить подтверждение.
11. Запустить `apply_profile.py`.
12. Провалидировать результат.
13. Вывести summary.

Создаваемые файлы

`.claude/settings.json`

`.claude/security-policy.json`

`.claude/rules/common/destructive-operations.md`

Ограничения

/harden не должен:

- запускать /bootstrap автоматически;
- перезаписывать custom policy;
- менять .claude/settings.local.json ;
- включать sandbox;
- требовать DCG.

Критерии приёмки

- Baseline применяется к новому проекту.
- Existing settings корректно merge.
- Dry-run ничего не меняет.

- Check сообщает drift.
- Повторный запуск идемпотентен.
- Команда явно сообщает conflicts.

Оценка

3–5 story points

Task 16. Добавить управляемый state и /

harden --remove

Цель

Позволить безопасно удалить только те settings, которые были добавлены claude-bootstrap .

Изменения

Создавать:

.claude/harden-state.json

Пример:

```
{
  "managedBy": "claude-bootstrap",
  "managedVersion": 1,
  "appliedProfile": "baseline",
  "insertedSettings": {
    "permissions.deny": [],
    "permissions.ask": [],
    "scalars": {}
  }
}
```

Команды

/harden --remove

/harden --check

Remove behavior

Удалять только:

- permissions entries, записанные в state;
- scalar settings, установленные bootstrap;
- bootstrap-managed sandbox settings;
- bootstrap-managed policy defaults, если они не изменялись пользователем.

Не удалять:

- пользовательские permission rules;
- пользовательские hooks;
- custom policy fields;

- .claude/rules/ .

Drift

Если managed entry был изменён пользователем:

- не удалять автоматически;
- сообщить conflict;
- потребовать --force .

Критерии приёмки

- Baseline можно безопасно удалить.
- Пользовательские entries сохраняются.
- Modified managed values не удаляются молча.
- State обновляется атомарно.
- Повторный remove ничего не меняет.

Оценка

3–5 story points

Task 17. Добавить strict profile

Цель

Добавить усиленный режим для проектов, где допустимо больше permission prompts.

Изменения

Создать:

plugin/hardening/profiles/strict.settings.json
plugin/hardening/defaults/strict-policy.json

Strict differences

Strict profile может:

- блокировать bypass permissions;
- отключать auto mode;
- переводить parser uncertainty в ask ;
- включать больше infrastructure checks;
- считать unknown environment high-risk;
- запрещать некоторые destructive database operations;
- не включать external guard; optional integration перенесена в Task 22.

Переходы

Поддержать:

baseline → strict

strict → baseline

strict → remove

Использовать managed state.

Ограничения

Strict не должен:

- блокировать весь Bash tool;
- делать arbitrary unknown command deny;
- зависеть от установленного DCG;
- автоматически включать sandbox.

Критерии приёмки

- Strict применяется поверх baseline architecture.
- Profile switching не оставляет старые managed entries.
- Unknown suspicious construct требует подтверждения.
- Пользовательские settings сохраняются.
- README содержит таблицу различий baseline/strict.

Оценка
3–5 story points

Task 18. Добавить preset конфигурации встроенного Claude Code Bash sandbox

Цель

Добавить отдельный opt-in preset, который включает встроенный Claude Code Bash sandbox через configuration overlay.
claude-bootstrap не реализует собственный sandbox runtime; он только управляет настройками в ``.claude/settings.json``.

Изменения

Создать только configuration overlay:

`plugin/hardening/profiles/sandbox.settings.json`

Пример целевой конфигурации:

```
{
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "allowUnsandboxedCommands": false,
    "credentials": {
      "files": [
        { "path": "~/.ssh", "mode": "deny" },
        { "path": "~/.aws/credentials", "mode": "deny" },
        { "path": "~/.kube/config", "mode": "deny" }
      ],
      "envVars": [
        { "name": "AWS_SECRET_ACCESS_KEY", "mode": "deny" },
        { "name": "NPM_TOKEN", "mode": "deny" },
        { "name": "PYPI_API_TOKEN", "mode": "deny" }
      ]
    }
  }
}
```

Команды

```
/harden --baseline --sandbox
/harden --strict --sandbox
/harden --remove-sandbox
```

Platform handling

На native Windows:

- не применять overlay частично;
- вывести понятную ошибку;
- предложить WSL2, container или поддерживаемую платформу Claude Code sandbox.

Перед применением

Сообщить, что встроенный sandbox может нарушить работу команд, которым нужны credentials или внешние ресурсы:

gh, kubectl, AWS CLI, package publication, private registry access.

Не входит в задачу

- собственная process isolation;
- собственная filesystem isolation;
- собственный network proxy;
- собственный container runtime;
- собственная credential isolation;
- собственная реализация Windows sandbox;
- runtime диагностика внутренней реализации Claude Code sandbox.

Критерии приёмки

- Sandbox включается только явно.
- Overlay применяется только через ``.claude/settings.json``.
- Existing sandbox settings не перезаписываются без conflict handling.
- Unsupported platform не получает частичную configuration.
- ``/harden --remove-sandbox`` удаляет только managed sandbox entries.
- ``/doctor`` показывает sandbox configuration status.

Оценка

3-5 story points

Task 19. Расширить /doctor hardening-проверками

Цель

Позволить диагностировать глобальную установку и состояние текущего проекта без изменения файлов.

Проверки

Добавить:

Python version
command_guard.py
hooks/guard modules
hardening profiles
policy schema
/harden skill
legacy hooks
duplicate hooks
project settings
project policy
managed state
profile drift
sandbox configuration

Sandbox configuration checks

/doctor проверяет только настройки sandbox:

- включён ли sandbox;
- поддерживается ли текущая платформа для выбранной configuration;
- установлен ли ``failIfUnavailable``;
- отключён ли ``allowUnsandboxedCommands``;
- настроены ли credential deny entries;
- отсутствуют ли опасные exclusions.

Runtime diagnostics

Диагностику фактического runtime sandbox, Seatbelt, bubblewrap, network behavior и platform internals делегировать штатной команде Claude Code ``/sandbox``.

Пример output

Hardening

=====

Python:	3.12.2 - OK
Command guard:	installed - OK
Profiles:	baseline, strict, sandbox - OK
Project policy:	baseline - OK
Permissions:	18 managed rules - OK
Legacy hooks:	none - OK
Sandbox config:	disabled
Sandbox runtime:	use Claude Code /sandbox
Overall:	OK

Требования

/doctor остаётся read-only.

Не исправлять автоматически:

- missing hooks;

- invalid policy;
- profile drift;
- sandbox conflicts.

Критерии приёмы

- Doctor видит новую architecture.
- Legacy installation определяется.
- Invalid policy отображается отдельно.
- Profile drift определяется.
- Sandbox configuration проверяется без runtime probing.
- Отсутствие Python не ломает остальные проверки.

Оценка

3 story points

Task 20. Расширить CI для hardening-инфраструктуры

Цель

Добавить автоматическую проверку собственной конфигурации и Python guard logic без тестирования внутреннего sandbox runtime Claude Code.

CI matrix

Pure Python tests:

ubuntu-latest

macos-latest

windows-latest

Python 3.10

Python 3.12

Bash integration tests:

ubuntu-latest

macos-latest

Checks

Добавить:

python -m compileall

python -m unittest

JSON validation

policy schema validation

hook integration tests

legacy-reference sentinel

duplicate permission sentinel

settings merge tests

managed state tests

sandbox overlay enable/disable tests

platform handling tests

custom settings preservation tests

unsafe-default sentinel

CI должен падать, если active hook configuration содержит:

block-no-verify.sh

block-large-files.sh

warn-secrets.sh

warn-debug-code.sh

Не тестировать

- внутреннюю реализацию Seatbelt;
- bubblewrap internals;
- network proxy internals;
- Claude Code sandbox process isolation;
- реальные cloud credentials.

Дополнительные проверки

- apply_profile.py --check на fixture projects;
- повторное применение profile;
- baseline -> strict -> baseline;
- sandbox overlay enable/disable;
- invalid settings rollback;
- secret redaction;
- guard execution timeout.

Критерии приёмки

- Python logic проверяется на трёх OS.
- Bash integration проверяется на Linux/macOS.
- Active legacy hooks запрещены sentinel-проверкой.
- Profile merge и removal покрыты CI.
- Sandbox tests проверяют только bootstrap-owned settings behavior.
- CI не требует сторонних Python dependencies.

Оценка

3-5 story points

Task 21. Обновить документацию hardening-релиза

Цель

Документировать новую модель безопасности, границы ответственности и ограничения

.

README

Добавить:

```
/bootstrap
/ Harden --baseline
/ Harden --strict
/ Harden --baseline --sandbox
/ Harden --check
/ Harden --remove
```

Разделение ответственности

```
Rules          -> behavioral guidance
Permissions    -> native static ask/deny
Hooks          -> project-specific contextual checks
Claude Code    -> command matching and sandbox runtime
External guard -> optional advanced analysis
```

Обязательные ограничения

Документировать:

- rules не являются security boundary;
- project settings можно изменить в checkout;
- enterprise enforcement требует managed settings;
- claude-bootstrap не реализует собственный sandbox;
- sandbox runtime принадлежит Claude Code;
- sandbox не поддерживается на native Windows, если это ограничение текущей платформы Claude Code;
- guard не выполняет полный Bash AST analysis;
- embedded scripts могут потребовать отдельного review;
- hardening не заменяет CI secret scanner;
- strict создаёт дополнительные prompts.

Дополнительно

Обновить:

- Hooks table;
- Installation;
- Quick Start;
- Customization;
- Doctor output;
- Changelog;

- version metadata.

Критерии приёмки

- Пользователь понимает различия baseline, strict и sandbox overlay.
- Документация не обещает абсолютную защиту.
- Документация не утверждает, что claude-bootstrap реализует собственный sandbox
- Описан rollback.
- Описаны platform limitations.
- Примеры команд соответствуют фактическому поведению.

Оценка

2-3 story points

Task 22. Optional follow-up: external guard integration point

Цель

Вынести DCG или другой внешний анализатор за пределы основного hardening release

Эта задача не блокирует baseline, strict, sandbox, /doctor, CI или документацию основного релиза.

Policy

Будущий optional policy может выглядеть так:

```
{
  "externalGuard": {
    "enabled": false,
    "command": "dcg",
    "timeoutMs": 1000
  }
}
```

Правила включения

- integration отключена по умолчанию;
- baseline не вызывает external guard;
- strict не зависит от external guard;
- sandbox overlay не зависит от external guard;
- external guard подключается только явно;
- задача реализуется только после анализа реальных false negatives.

Выполнение

Порядок решений:

1. Internal high-confidence deny.
2. Internal context rules.
3. External guard, если явно включён.
4. Объединение решений по приоритету.

Инварианты

- external verdict не может переопределить internal deny;
- timeout не должен зависеть;
- unknown output не считается allow;
- command path должен быть configurable;
- stdout/stderr внешнего процесса необходимо ограничить;
- secrets не должны попадать в debug logs.

Ограничения

Конкретную поддержку DCG реализовывать только после проверки его актуального CLI contract.

Допускается сначала создать generic adapter interface и fake adapter для тестов.

Критерии приёмки

- External guard полностью отключён по умолчанию.

- Отсутствующий executable не ломает baseline или strict.
- Timeout корректно обрабатывается.
- Внешнее решение не может переопределить internal deny.
- Adapter покрыт tests.

Оценка

Optional follow-up, 3-5 story points

Рекомендуемый состав релиза

Release foundation

Можно выпускать после задач:

1. Destructive operations rule
2. Baseline permissions template
3. Settings merge
4. Security policy schema
5. block-no-verify fix

Результат: rules и безопасное управление project permissions без нового runtime guard.

Release command guard

Добавить:

6. Guard foundation
7. Git rules
8. Filesystem rules
9. Infrastructure/publication rules
10. Large file policy
11. PreToolUse secrets
12. PostToolUse warnings
13. Unified hook configuration
14. Installer migration

Результат: единый контекстный hook вместо набора Bash-скриптов.

Release hardening UX

Добавить:

15. /harden baseline
16. Managed state/remove
17. Strict profile
18. Claude Code Bash sandbox configuration preset
19. Doctor configuration diagnostics
20. CI for owned configuration and guard logic
21. Documentation

Результат: законченный пользовательский hardening workflow без собственного sand box runtime.

Optional follow-up

22. External guard integration point

Эта задача не должна блокировать основной hardening release.

Общий Definition of Done релиза

- [] Rules объясняют безопасный workflow.
- [] Baseline permissions применяются безопасным merge.
- [] /harden не изменяет пользовательские settings молча.
- [] Повторное применение идемпотентно.
- [] Managed settings можно удалить.
- [] Git bypass hooks блокируются без false positives.

- [] Force push protected branch блокируется.
- [] Dangerous filesystem commands блокируются.
- [] Legitimate destructive operations используют ask.
- [] Production context усиливает решение.
- [] High-confidence secrets проверяются до записи.
- [] Large file policy не мешает постепенному рефакторингу.
- [] Legacy hooks корректно мигрируются.
- [] Sandbox включается только явно через Claude Code settings overlay.
- [] claude-bootstrap не реализует собственный sandbox runtime.
- [] Doctor диагностирует установку и project state.
- [] Doctor делегирует runtime sandbox диагностику Claude Code /sandbox.
- [] Python tests выполняются на Linux, macOS и Windows.
- [] Документация явно описывает ограничения.
- [] External guard не является обязательной зависимостью.